Agent-Guided Reinforcement Learning for Robot Policy Improvement

Weijun Dong[1,‡] **Carl Qi**[2,3] **Mingyuan Zhang**[4] **Manikantan Nambi**[3]
Lillian J. Ratliff[3,5] **Jeroen Vanbaar**[3,†] **Dylan Randle**[3,†] **Jose Barreiros**[3,†]

[1] UC San Diego [2] UT Austin [3] Amazon [4] Northeastern University [5] University of Washington

Abstract: Reinforcement learning (RL) over generalist robot policies is a promising path to advance robotic manipulation, but it is bottlenecked by low sample efficiency: the RL learner must discover how to correct its own suboptimal behaviors through costly uninformed exploration. Meanwhile, modern foundation models can provide rich priors over both successful behaviors and counterfactual actions, supplying exactly the knowledge that the RL algorithm itself lacks. We introduce a novel framework that turns this prior into action-level guidance for reinforcement learning. When a rollout fails, a tool-using reasoning agent analyzes the trajectory, identifies states where actions can be improved, and produces action proposals for them. These diagnoses run asynchronously alongside training without blocking online rollouts. We incorporate the proposed actions into policy training via a soft imitation objective alongside the RL loss, steering subsequent exploration toward the reasoning agent's suggestions. We evaluate on ten LIBERO manipulation tasks, where our method achieves $6.9\times$ the normalized success throughput of the strongest pure-RL baseline within a 400-episode interaction budget. Moreover, our RL framework outperforms directly imitating the agent's proposals, especially when the agent is weaker.

Keywords: Reinforcement Learning, Agentic Robot Learning, Manipulation

1 Introduction

Generalist robot policies trained on large and diverse robot data have shown strong generalization across a wide range of manipulation tasks and scenes [1, 2, 3, 4, 5]. For real-world deployment, however, generalization alone is insufficient. High reliability and high execution efficiency are equally important. Real-world reinforcement learning (RL) on top of a pretrained generalist policy is a promising route toward this goal [6, 7, 8, 9]. Since the policy learns from its own interaction with the environment, training naturally concentrates on the situations where the policy fails and drives it to act correctly in them, offering a high performance ceiling.

However, the RL algorithm operates without any understanding of the physical world. When a rollout fails, it cannot infer what went wrong or what behavior would lead to success, and improvement must come entirely from exploration and reward-driven gradient updates. This makes current methods highly sample-inefficient, a serious bottleneck when real-world interaction is expensive.

On the other hand, a recent line of work has demonstrated that foundation-model agents can control robots directly, solving a range of tasks zero-shot by reasoning about the scene, formulating plans, and issuing actions or orchestrating lower-level skills [10, 11, 12, 13]. While scaling these approaches to high success rate and execution efficiency remains an open question, these results demonstrate that foundation models carry a strong understanding of the physical world and can translate it into effective action-level decisions. This is exactly the capability that RL lacks. A natural question is: *can this prior be effectively incorporated into the RL process to guide exploration?*

[†]Equal advising. ‡ Weijun Dong, Carl Qi, and Mingyuan Zhang were interns at Amazon during this work.

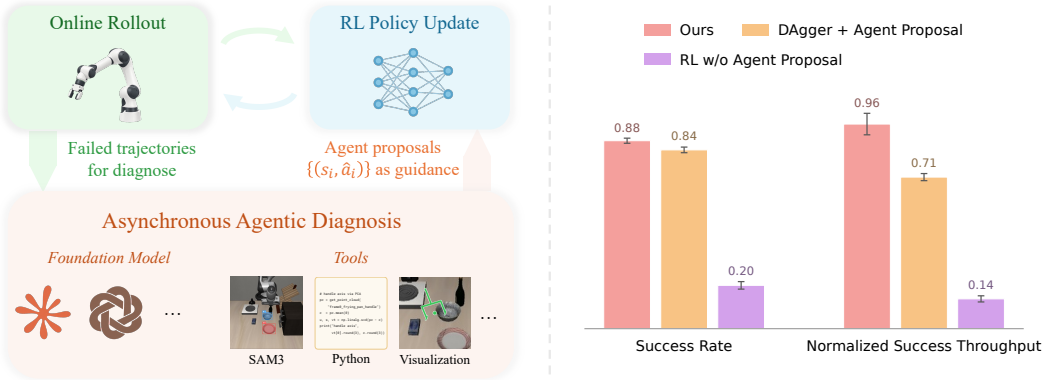


Figure 1: **Overview.** Left: our online reinforcement learning loop, in which a foundation-model agent asynchronously diagnoses failed trajectories and returns action proposals that guide policy updates. Right: on ten LIBERO tasks, the agent's guidance lifts success rate far above RL without it, and RL attains higher execution efficiency than DAgger under the same agent.

We answer this question by using a tool-using foundation-model agent to guide RL during training. When a rollout fails, the agent analyzes the collected trajectory asynchronously, identifies states where the original actions can be improved, and produces an action proposal at each of them. As the policy evolves and encounters new failure modes, the agent produces proposals for the newly visited states. These proposals enter the policy update through a soft auxiliary imitation loss, biasing the policy toward the agent's suggested behavior and thereby guiding subsequent exploration. When the proposals are imperfect, the RL objective remains free to optimize further. In this way, the foundation model's understanding of the physical world is effectively brought into the RL process as an actionable prior, enabling strong performance within a limited interaction budget. Our contributions are summarized as follows:

- We propose a tool-using agentic pipeline in which a foundation-model agent analyzes collected trajectories offline and produces action-level proposals at states where the behavior can be improved.
- We introduce an online RL framework in which rollout collection, asynchronous agent analysis, and policy optimization form a continuous loop. The agent's action proposals are incorporated into policy updates through a soft imitation loss, effectively channeling the foundation model's knowledge into RL learning.
- We evaluate on ten LIBERO tasks, where our method reaches $6.9\times$ the normalized success throughput of the best pure-RL baseline within a 400-episode budget. Moreover, RL over these proposals outperforms directly imitating them on average, and the gap widens under a weaker guiding model, showing that RL can improve beyond imperfect proposals.

2 Related work

Foundation-Model Agents for Robotic Manipulation. A growing line of work places a foundation-model agent directly in the control loop. Given a task and the current observation, the agent reasons about the scene, generates a plan, and either issues actions itself or orchestrates lower-level modules and skills, closing the loop over the outcomes it observes [12, 10, 13, 11, 14]. Some of these systems also improve over time by having the agent edit its own artifacts: rewriting behavior code from rollout logs [15], growing a reusable skill library [16], or self-organizing a computation graph that is later executed without the agent [17]. Across these efforts, under limited online interaction budgets, obtaining a policy that achieves high success rates with smooth, fast execution remains an open question. Our results in simulation (Section 4.2) suggest a promising step in this direction. Beyond authoring code artifacts, Xiao et al. [15] further use the agent to rewrite RL training code across full iterations, shaping the policy indirectly through algorithm-level edits. In contrast, our

method feeds the agent's action proposals directly into the RL objective, allowing the foundation model's behavioral prior to shape the policy more directly and more frequently.

RL Post-training of Robotic Policies. Recent work improves pretrained generative robot policies through reinforcement learning, either by directly adapting the policy parameters or by steering a frozen policy. The former includes value-guided and policy-gradient methods for diffusion and flow policies [9, 18, 19, 20, 21, 22, 23], as well as approaches that adapt an auxiliary policy online and distill it back into the base model [24]. The latter keeps the pretrained generative policy fixed and searches within its latent or noise space: reranking latent proposals using learned values [25], training actors directly over the policy's noise variables [26], steering generation through constrained latent perturbations [27], or inverting demonstrated actions into noise-space RL training data [28].

A separate line of work targets the sample efficiency of real-world RL through interventions during rollouts. Human-in-the-loop methods let a human intervene to correct the policy [29, 30], at the cost of continuous human attention throughout training. Deng et al. [31] learn a recovery module that returns the robot to a recoverable earlier state, reducing the human intervention required. Closer to us, Chen et al. [32] replace the human intervener entirely with a tool-using agent that issues corrective waypoints when the policy deviates from expert behavior; each call blocks the rollout, so the agent's inference latency and reasoning depth trade against training throughput. Our method instead runs the agent post-hoc on completed failure episodes, entirely off the rollout loop, allowing many diagnoses to run in parallel and giving the agent ample room for extended reasoning.

3 Method

We propose a novel framework that improves a base policy through online RL, guided by a foundation-model reasoning agent. The system runs as a continuous loop: the policy collects rollouts in the environment, a reasoning agent analyzes failed episodes asynchronously in parallel, and policy optimization consumes both environment returns and the agent's proposals. Section 3.1 introduces the agentic diagnosis pipeline, and Section 3.2 describes how these proposals are integrated into the RL loop.

3.1 Agentic Diagnosis of Failed Rollouts

In this section, we introduce an agentic diagnosis pipeline to turn failed rollouts into corrective guidance for RL. A reasoning agent receives a complete failed trajectory, analyzes it to understand why the task failed, and proposes better actions at states where it believes improvement is possible. The agent operates through multi-turn reasoning with tool use, iteratively inspecting observations, computing quantities of interest, and calling external specialist models before arriving at its proposals. This analysis is conducted post-hoc over the recorded episode without intervening in the environment. The underlying foundation model is interchangeable (e.g., Claude or ChatGPT).

Input. The agent receives a complete failed trajectory, comprising the natural-language task instruction, multi-camera RGBD streams, the commanded action log, the end-effector pose trajectory, and camera calibration. All of these are stored as read-only files that the agent can access at any point during its reasoning. No privileged simulator state is exposed.

Output. Through its reasoning, the agent identifies states where it judges the rollout to be deviating from successful task completion and where it can propose a better action. At each such state, the agent outputs a short-term target end-effector pose and gripper state. This proposal is then converted into a proposed action chunk $\hat{a}$ ready to be consumed by the RL algorithm (see Appendix A.1).

Tools. Beyond standard file access for browsing frames, logs, and task specifications, the agent has access to the capability tools listed in Table 1, which fall into three groups that together form a closed loop: the agent can query external specialist models for perception and grasp generation, perform numerical computation, and verify its hypotheses through visualization, iterating over multiple turns to arrive at its final proposals.

Table 1: Capability tools available to the diagnosis agent.

Group	Tool Name	Function
Computation	`run_python`	Executes arbitrary agent-written Python (with `numpy`) in an isolated sandbox.
External model calls	`locate_object`	Given a text prompt and a specified image, segments the object using SAM3 [33] and returns a mask overlay image, per-instance confidence scores, and world-frame 3D point clouds.
	`plan_grasp`	Given a frame and an object point cloud from `locate_object`, generates scored 6-DOF grasp candidates in the world frame using Contact-GraspNet [34].
Visualization	`overlay_points`	Given a frame and a set of world-frame 3D points, overlays the visible ones as numbered markers on all camera images of that frame.
	`visualize_traj`	Given a frame and a target pose, draws the end-effector center's predicted path toward it on the third-person and wrist images.
	`visualize_gripper_pose`	Given a frame and a pose, draws the gripper's shape exactly projected onto the third-person image, as a full view and a magnified crop.

Algorithm 1 Reinforcement Learning Guided by Agent-Proposed Actions.

Require: frozen base policy π_{base}; reasoning agent AGENTPROPOSE

initialize critic Q_ψ, refinement policy π_θ, replay buffer $\mathcal{B} \leftarrow \emptyset$
for episode $e = 1, 2, \ldots$ **do**
  $\tau \leftarrow \emptyset$
  for $t = 0, C, 2C, \ldots$ until the episode ends **do**
    sample base reference chunk $\tilde{a}_{t:t+C-1} \sim \pi_{\text{base}}(\cdot \mid s_t)$
    $a_{t:t+C-1} \sim \pi_\theta(\cdot \mid s_t, \tilde{a}_{t:t+C-1})$
    execute $a_{t:t+C-1}$, observe rewards $r_{t:t+C-1}$ and next state s_{t+C}
    append $\langle s_t,\ a_{t:t+C-1},\ \tilde{a}_{t:t+C-1},\ \hat{a}_{t:t+C-1} = \emptyset,\ r_{t:t+C-1},\ s_{t+C} \rangle$ to τ
  end for
  if τ failed **then** $\tau \leftarrow$ AGENTPROPOSE(τ) ▷ agent diagnosis fills proposals $\hat{a}$ at chosen states
  $\mathcal{B} \leftarrow \mathcal{B} \cup \tau$
  for $g = 1, \ldots, G \cdot |\tau|$ **do**
    sample batch $b \sim \mathcal{B}$ of transitions $\langle s,\ a,\ \tilde{a},\ \hat{a},\ r_{1:C},\ s' \rangle$
    compute target $\hat{Q} \leftarrow \sum_{j=1}^{C} \gamma^{j-1} r_j + \gamma^C\, \mathbb{E}_{a' \sim \pi_\theta(\cdot \mid s', \tilde{a}')} \left[Q_{\psi'}(s', a') \right]$
    train critic: $\mathcal{L}_Q(\psi) = \mathbb{E}_b \left[\left(\hat{Q} - Q_\psi(s, a) \right)^2 \right]$
    train policy: one step on $\mathcal{L}_\pi(\theta)$, with $a_\theta \sim \pi_\theta(\cdot \mid s, \tilde{a})$

$$\mathcal{L}_\pi(\theta) \;=\; \mathbb{E}_b \left[-\frac{Q_\psi(s, a_\theta)}{\bar{Q}_b} \;+\; \begin{cases} \beta_{\text{prop}} \left\| a_\theta - \hat{a} \right\|_2^2 & \text{if } \hat{a} \neq \emptyset \\ \beta_{\text{base}} \left\| a_\theta - \tilde{a} \right\|_2^2 & \text{otherwise} \end{cases} \right]$$

  end for
end for

3.2 Reinforcement Learning with Agent-Proposed Actions

Our goal is to improve a pretrained policy through online RL, guided by the agentic diagnosis of Section 3.1. The agent's corrective proposals enter the actor loss as auxiliary anchor targets, steering policy improvement toward fixes for observed failures. Algorithm 1 summarizes the procedure.

Problem formulation. We consider an MDP $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$ with reward $r = -1$ at every time step and $r = 0$ upon task success, at which point the episode terminates. We assume access to a frozen pretrained base policy π_{base} that outputs action chunks of length C. Our goal is to learn a refinement policy $\pi_\theta(\cdot \mid s, \tilde{a})$: given the current state s and a reference chunk $\tilde{a} \sim \pi_{\text{base}}(\cdot \mid s)$, it outputs the action chunk a to execute.

Base RL algorithm. Our underlying RL algorithm follows RL Token [7]. Specifically, we keep π_{base} frozen and train a lightweight refinement policy π_θ, parameterized as a Gaussian with a small fixed standard deviation. During online episode rollouts, the robot executes one action chunk of length C at each decision step. In a warmup phase (omitted from Algorithm 1 for brevity), reference chunks $\tilde{a}_{t:t+C-1} \sim \pi_{\text{base}}(\cdot \mid s_t)$ are sampled and executed directly; after warmup, the robot instead executes $a_{t:t+C-1} \sim \pi_\theta(\cdot \mid s_t, \tilde{a}_{t:t+C-1})$.

The critic $Q_\psi(s, a)$ operates at the chunk level: it regresses on the executed chunk a with TD target $\hat{Q} = \sum_{j=1}^{C} \gamma^{j-1} r_j + \gamma^C\ \mathbb{E}_{a' \sim \pi_\theta(\cdot \mid s', \tilde{a}')}[Q_{\psi'}(s', a')]$, bootstrapping from π_θ at the next chunk

boundary. To softly constrain exploration to the neighborhood of the base policy's reference actions, the actor update draws $a_\theta \sim \pi_\theta(\cdot \mid s, \tilde{a})$ and minimizes $\mathcal{L}_\pi(\theta) = \mathbb{E}_b[-Q_\psi(s, a_\theta)/\bar{Q}_b + \beta_{\text{base}} \|a_\theta - \tilde{a}\|_2^2]$, where $\tilde{a}$ is the reference chunk and $\bar{Q}_b$ is a detached batch normalizer for the Q term.

Integrating agent proposals. When an episode fails, we invoke the diagnosis agent of Section 3.1 on the recorded trajectory. It selects a subset of chunk-boundary states and proposes a corrective action chunk $\hat{a}$ at each. The diagnosis is conducted post-hoc over the completed trajectory, and the proposed chunks are not executed in the environment. We therefore dispatch diagnoses for multiple episodes in parallel; whenever one completes, its proposals are written into the replay buffer and take effect in subsequent policy updates. This design ensures that the agent's reasoning time does not block online data collection or training.

The proposals enter training through the actor loss, steering the policy toward the agent's corrections. To this end, we make its auxiliary behavior cloning term state-dependent, so that the actor minimizes

$$\mathcal{L}_\pi(\theta) = \mathop{\mathbb{E}}_{b}\left[-\frac{Q_\psi(s, a_\theta)}{\bar{Q}_b} + \begin{cases} \beta_{\text{prop}} \|a_\theta - \hat{a}\|_2^2 & \text{if the agent proposed a correction at } s \\ \beta_{\text{base}} \|a_\theta - \tilde{a}\|_2^2 & \text{otherwise} \end{cases}\right]. \quad (1)$$

We set $\beta_{\text{prop}} = 0.1$ and $\beta_{\text{base}} = 0.01$, since the base reference serves only as a soft regularizer that keeps exploration local, whereas a proposal is a more trusted target that the policy is drawn toward more strongly. Meanwhile, the critic term remains active, so if a proposal turns out to lower returns, the value gradient drives the policy away from it.

4 Experiments

In this section, we aim to answer the following research questions: i) how much does agentic guidance improve the sample efficiency of online RL? ii) given the same agent proposals, what does RL add over directly imitating them? iii) how does our training-time guidance approach relate to zero-shot agentic control? iv) how does the diagnosis agent use its tools, and how much do they matter for downstream policy performance? We first describe the experimental setup in Section 4.1, then address i)–iii) in Section 4.2 and iv) in Section 4.3.

4.1 Experimental Setup

Tasks and environment. We evaluate on the LIBERO benchmark [35], a suite of simulated tabletop manipulation tasks. Following FRS [28], we use their ten hard LIBERO-90 tasks (see Figure 2 for the full task list). The online interaction budget is 400 episodes per task. Each episode has a horizon of 400 environment steps, and episodes that exceed this limit are counted as failures. At every step the policy observes two RGB camera views (exocentric scene and wrist) together with proprioceptive state, and outputs 7-dimensional actions in chunks of length 10.

Base policy. Our base policy is $\pi_{0.5}$ [2], a flow-matching vision-language-action model. Following FRS [28], we use OpenPI's off-the-shelf $\pi_{0.5}$-LIBERO checkpoint, which is trained on all LIBERO task suites except the LIBERO-90 tasks and is therefore out-of-distribution on our ten evaluation tasks. It achieves near-zero success on the ten tasks.

Evaluation metrics. Our main metric is *success throughput*: the number of successful evaluation episodes divided by the total steps across all evaluation episodes, capturing both success rate and completion speed in a single number. We normalize per task by a $\pi_{0.5}$ variant fine-tuned on LIBERO-90, so that 1.0 corresponds to solving the task on every attempt as quickly as this in-distribution expert.

4.2 Main Results

Comparison to online policy-improvement baselines. We compare against two families of training-based methods that build on the same frozen base policy.



Figure 2: Main results on ten LIBERO tasks. We report normalized success throughput over the online interaction budget of 400 episodes, shown per task and averaged over all ten tasks (left). Curves show the mean over six seeds and shaded regions denote the standard error.

Table 2: *RL + Agent Proposal* versus *DAgger + Agent Proposal*, compared separately under a stronger (Fable 5.1) and a weaker (Opus 4.8) agent model. We report the mean success rate (↑) and the mean length of successful episodes (in environment steps, ↓), averaged over all tasks and broken down per task. Within each agent model, the better of RL and DAgger on each column is in bold.

	Method	Overall	T1	T3	T18	T25	T36	T37	T49	T63	T69	T78
Success Rate ↑	RL (Fable 5.1)	**0.88**	0.74	0.87	**0.95**	**0.95**	**0.93**	**0.95**	**0.97**	**0.64**	**0.99**	**0.80**
	DAgger (Fable 5.1)	0.84	**0.78**	**0.95**	0.89	0.90	**0.95**	0.65	**0.98**	0.59	**0.98**	0.71
	RL (Opus 4.8)	**0.58**	**0.17**	**0.01**	**0.83**	**0.86**	**0.80**	**0.72**	**0.86**	**0.29**	**0.93**	**0.36**
	DAgger (Opus 4.8)	0.26	0.04	**0.00**	0.53	0.36	0.19	0.31	0.53	0.01	0.63	0.01
Successful Episode Length ↓	RL (Fable 5.1)	**146**	**249**	**223**	**119**	**105**	**127**	**126**	**114**	**229**	**74**	**98**
	DAgger (Fable 5.1)	179	259	**219**	169	168	157	198	150	263	91	118
	RL (Opus 4.8)	**167**	284	338	**120**	**122**	**137**	**130**	**107**	**228**	**76**	128
	DAgger (Opus 4.8)	173	**275**	**295**	129	138	188	137	131	241	105	**88**

- RL that improves the base policy. **RL Token** is our full algorithm with the agent proposals removed. This recovers the RL algorithm of RLT [7], and drops its demonstration-trained RL-token readout, teleoperator interventions during rollouts, and critical-phase resets. **DSRL** [26] instead steers the base policy through latent-space reinforcement learning.
- Imitation ablation of our method that removes the RL objective from Eq. (1). After a warmup phase that initializes the refinement network to the base policy, the actor imitates the agent's corrective proposals as its sole training signal, with no base-policy anchoring and no value-based improvement. This resembles DAgger [36] with the diagnosis agent in place of an expert; we refer to it as **DAgger + Agent Proposal**. This baseline and our method use the same agent model (Fable 5.1 with high reasoning effort) for fair comparison.

Figure 2 summarizes our main results. Our method reaches an average normalized success throughput of 0.96 within the 400-episode budget. This is nearly 7× that of the best pure-RL baseline (RL Token, 0.14), showing the substantial impact of agentic guidance. It is also 35% higher than DAgger + Agent Proposal (0.71), demonstrating the added value of RL over imitating the agent's proposals alone. On Tasks 18, 25, 36, 49, and 69 our normalized success throughput exceeds 1.0 (1.16 to 1.39), meaning the learned policy solves these tasks faster than the in-distribution fine-tuned $\pi_{0.5}$.

RL versus DAgger under agents of different strength. We compare our method against the DAgger ablation while varying the agent model that supplies the corrections, using a stronger model (Fable 5.1) and a weaker one (Opus 4.8); results are reported in Table 2. With the stronger agent, DAgger attains a success rate close to RL on average (0.84 vs. 0.88) and even surpasses it on Tasks 1 and 3, consistent with the theoretical guarantee that interactive imitation recovers near-expert performance when the corrective labels are near-optimal [36, 37]. RL nonetheless completes successful episodes in 18% fewer steps on average (146 vs. 179), because optimizing discounted return en-

Table 3: Task performance of our method and CaP-Agent0. Our policy is evaluated at its final checkpoint after the 400-episode online budget. CaP-Agent0 is zero-shot and requires no training.

	CaP-Agent0	Ours
Success Rate (within 400 steps) ↑	0.00	0.88
Success Rate (within 5000 steps) ↑	0.51	–
Successful Episode Length ↓	2589	146

Table 4: Foundation-model (Fable 5.1) usage of our method and CaP-Agent0. We report per-diagnosis cost during training for our method and per-trial cost at evaluation for CaP-Agent0.

CaP-Agent0 (per trial in evaluation)	Output Tokens 455K	Wall Clock 28 min
Ours (per diagnosis in training)	Output Tokens 36K	

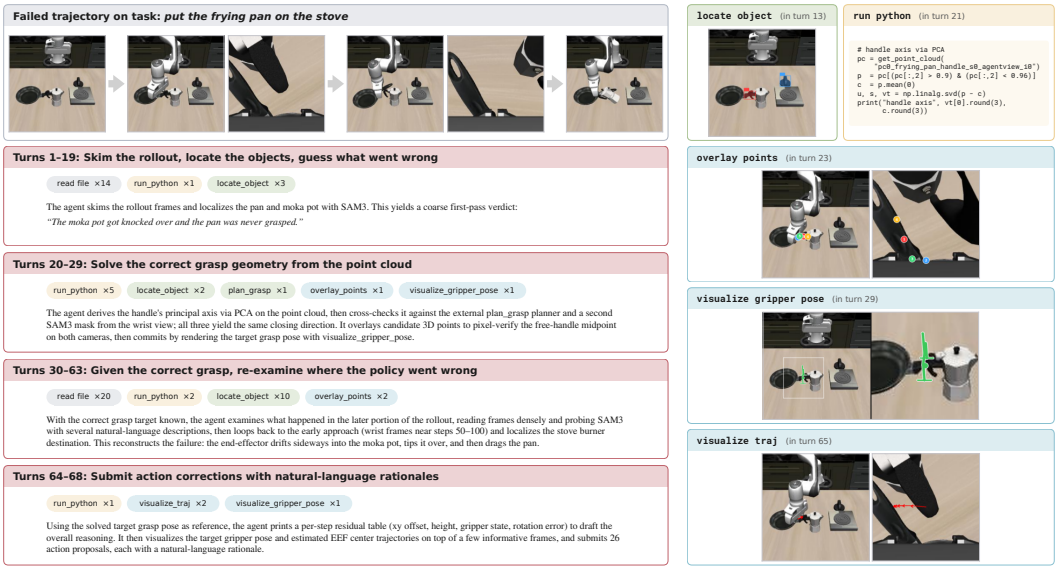


Figure 3: A representative diagnosis on a failed rollout of task 18, *put the frying pan on the stove*. The gripper misses the pan handle, topples the moka pot, and times out (top left). We summarize the agent's diagnosis process and tool-use counts (bottom left), and show examples of tool use (right).

courages the policy to reach the goal quickly. With the weaker agent, RL leads in success rate by a wide margin (0.58 vs. 0.26). Online rollouts let the policy discover better actions through exploration, and the learned critic then drives it away from poor proposed actions. In contrast, DAgger is fundamentally limited by the quality of the proposals it imitates.

Comparison with CaP-Agent0. We include **CaP-Agent0** [12], an agent that controls the robot zero-shot through iterative code generation, as a reference for what direct agentic control can achieve on our tasks. It uses the same foundation model (Fable 5.1) and a perception-and-compute toolset at least as strong as that of our diagnosis agent. As shown in Table 3, CaP-Agent0 solves roughly half the trials given a generous 5000-step horizon, confirming that foundation models carry a meaningful manipulation prior, though successful episodes average 2589 steps. The per-trial cost is also substantial (Table 4): 455K output tokens and 28 minutes of wall-clock time. Because the foundation model reasons inside the control loop, this cost recurs on every trial; on a physical robot, execution is further slowed by the agent's reasoning latency.

As an alternative approach, our method confines all foundation-model usage to the training phase, where the agent analyzes collected trajectories offline at an average of 36K output tokens per diagnosis. Because multiple diagnoses run in parallel and asynchronously with trajectory rollouts, the agent's reasoning latency does not slow down training. After the 400-episode online budget the deployed policy is entirely agent-free, achieving 0.88 success rate within a 400-step horizon and averaging 146 steps per successful episode.

Table 5: Effect of removing tool groups on final policy performance. We report normalized success throughput (mean $\pm$ standard error over six seeds).

All tools	w/o numeric computation	w/o external models	w/o visualization	w/o any tools
$\mathbf{0.96} \pm 0.05$	0.93 ± 0.03	0.88 ± 0.03	0.92 ± 0.03	0.75 ± 0.02

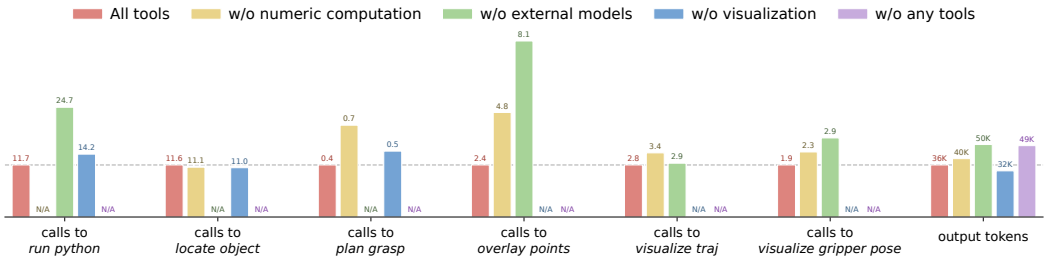


Figure 4: Average tool-call counts and output tokens per diagnosis for the full agent and each ablation variant. Each metric is normalized to the *All tools* agent, whose value is marked by the grey dashed line. Bars are annotated with absolute values.

4.3 Understanding the Diagnosis Agent

How the agent diagnoses a rollout. Figure 3 illustrates a representative diagnosis. The agent segments the pan handle with `locate_object` to obtain a 3D point cloud, computes its principal axis via PCA in `run_python`, and cross-checks the grasp orientation against the external `plan_grasp` planner. It then pixel-verifies the target point on both cameras with `overlay_points` and renders the committed grasp pose with `visualize_gripper_pose`. With the correct pose established, the agent re-examines the full trajectory to pinpoint where the end-effector deviates from the target and submits corrective action proposals at 26 states.

Ablation of the agent's tools. We ablate each of the three tool groups in Table 1 from the diagnosis agent while keeping everything else identical. As shown in Table 5, removing any single group leads to a mild throughput drop (at most 0.08, for external models), suggesting that the agent is relatively robust to partial tool removal. Figure 4 shows why: when a tool group is unavailable, the agent compensates by increasing its use of the remaining tools. For example, removing the external perception models roughly doubles the agent's `run_python` calls ($11.7 \rightarrow 24.7$) and more than triples its `overlay_points` calls ($2.4 \rightarrow 8.1$), as it writes custom code to localize objects and overlays the results for visual verification. This substitution avoids a large performance drop, though output tokens per diagnosis rise by about 40% due to the additional reasoning effort. However, removing every capability tool at once leaves the agent with only read-only file access and forces it to reason in text alone. Throughput drops sharply to 0.75, far below any single-group removal, confirming that the tools collectively provide grounding that text-only reasoning cannot fully recover.

5 Conclusion

Sample efficiency remains a key bottleneck for reinforcement learning over pretrained robot policies, because the algorithm cannot reason about why a rollout failed or how to correct it. Foundation models carry exactly this understanding and can translate it into action-level decisions. We introduce a framework that channels this prior into RL by having a tool-using reasoning agent analyze failed trajectories asynchronously and propose corrective actions, which enter policy updates through a soft imitation loss. On ten LIBERO tasks, this guidance yields roughly $7\times$ the normalized success throughput of the strongest pure-RL baseline. RL over imperfect agent proposals also substantially outperforms directly imitating them, and the gap widens as the guiding model becomes weaker. Future work will transfer the full pipeline to real-world reinforcement learning on physical robots.

References

[1] G. R. Team, A. Abdolmaleki, S. Abeyruwan, J. Ainslie, J.-B. Alayrac, M. G. Arenas, A. Balakrishna, N. Batchelor, A. Bewley, J. Bingham, et al. Gemini robotics 1.5: Pushing the frontier of generalist robots with advanced embodied reasoning, thinking, and motion transfer. *arXiv preprint arXiv:2510.03342*, 2025.

[2] P. Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, et al. $\pi_{0.5}$: A vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.

[3] J. Bjorck, F. Castañeda, N. Cherniadev, X. Da, R. Ding, L. Fan, Y. Fang, D. Fox, F. Hu, S. Huang, et al. Gr00t n1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025.

[4] P. Intelligence, B. Ai, A. Amin, R. Aniceto, A. Balakrishna, G. Balke, K. Black, G. Bokinsky, S. Cao, T. Charbonnier, et al. $\pi_{0.7}$: A steerable generalist robotic foundation model with emergent capabilities. *arXiv preprint arXiv:2604.15483*, 2026.

[5] S. Ye, Y. Ge, K. Zheng, S. Gao, S. Yu, G. Kurian, S. Indupuru, Y. L. Tan, C. Zhu, J. Xiang, et al. World action models are zero-shot policies. *arXiv preprint arXiv:2602.15922*, 2026.

[6] Y. Guo, J. Zhang, X. Chen, X. Ji, Y.-J. Wang, Y. Hu, and J. Chen. Improving vision-language-action model with online reinforcement learning. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 15665–15672. IEEE, 2025.

[7] C. Xu, J. T. Springenberg, M. Equi, A. Amin, A. Esmail, S. Levine, and L. Ke. Rl token: Bootstrapping online rl with vision-language-action models. *arXiv preprint arXiv:2604.23073*, 2026.

[8] P. Intelligence. $\pi^*_{0.6}$: a VLA That Learns From Experience, 2025. URL https://arxiv.org/abs/2511.14759.

[9] Y. Li, X. Ma, J. Xu, Y. Cui, Z. Cui, Z. Han, L. Huang, T. Kong, Y. Liu, H. Niu, et al. Gr-rl: Going dexterous and precise for long-horizon robotic manipulation. *arXiv preprint arXiv:2512.01801*, 2025.

[10] J. Shi, R. Yang, K. Chao, B. S. Wan, Y. S. Shao, J. Lei, J. Qian, L. Le, P. Chaudhari, K. Daniilidis, et al. Maestro: Orchestrating robotics modules with vision-language models for zero-shot generalist robots. In *NeurIPS 2025 Workshop on Space in Vision, Language, and Embodied AI*, 2025.

[11] J. Su, Y. Zheng, M. Yan, L. Yi, Z. Zhang, and H. Wang. GPT 6 Astra as an embodied policy. Technical report and code, 2026. URL https://github.com/anonymous-report-421/eval-of-gpt-6-astra-as-policy.

[12] L. Fu, J. Yu, K. El-Refai, E. Kou, H. Xue, H. Huang, W. Xiao, G. Wang, D. Niu, F.-F. Li, et al. Cap-x: A framework for benchmarking and improving coding agents for robot manipulation. *arXiv preprint arXiv:2603.22435*, 2026.

[13] L. Galanti, D. Shah, and T. Dao. Addressing the orchestration gap in generalist robots via physical agency. *arXiv preprint arXiv:2607.21725*, 2026.

[14] H. Hu, P. Sundaresan, J. Gao, and D. Sadigh. Via: Visual interface agent for robot control. *arXiv preprint arXiv:2607.11119*, 2026.

[15] W. Xiao, J. Xie, T. Zhang, H. Lin, L. Fu, H. Xue, J. Lu, Y. Yang, C. Dai, Z. Wang, et al. Enpire: Agentic robot policy self-improvement in the real world. *arXiv preprint arXiv:2606.19980*, 2026.

[16] R. Lu, Y. Wu, E. Kou, L. Fu, W. Xiao, A. Mandlekar, Y. Xu, G. Shi, K. Goldberg, A. Chen, et al. Aspire: Agentic/skills discovery for robotics. *arXiv preprint arXiv:2607.00272*, 2026.

[17] K. Chen, S. Xie, L. Fu, J. Yu, W. Pacini, S. Bajamahal, H. Kim, J. Drake, D. Kim, H. Xue, et al. Gap: A graph-as-policy multi-agent self-learning harness for variational automation tasks. *arXiv preprint arXiv:2607.05369*, 2026.

[18] M. Psenka, A. Escontrela, P. Abbeel, and Y. Ma. Learning a diffusion model policy from rewards via Q-score matching. In *International Conference on Machine Learning (ICML)*, 2024.

[19] A. Z. Ren, J. Lidard, L. L. Ankile, A. Simeonov, P. Agrawal, A. Majumdar, B. Burchfiel, H. Dai, and M. Simchowitz. Diffusion policy policy optimization. *arXiv preprint arXiv:2409.00588*, 2024.

[20] S. Park, Q. Li, and S. Levine. Flow Q-learning. In *International Conference on Machine Learning (ICML)*, 2025.

[21] T. Zhang, C. Yu, S. Su, and Y. Wang. ReinFlow: Fine-tuning flow matching policy with online reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[22] D. McAllister, S. Ge, B. Yi, C. M. Kim, E. Weber, H. Choi, H. Feng, and A. Kanazawa. Flow matching policy gradients. *International Conference on Learning Representation (ICLR)*, 2026.

[23] K. Lei, H. Li, D. Yu, Z. Wei, L. Guo, Z. Jiang, Z. Wang, S. Liang, and H. Xu. Rl-100: Performant robotic manipulation with real-world reinforcement learning. *arXiv preprint arXiv:2510.14830*, 2025.

[24] W. Xiao, H. Lin, A. Peng, H. Xue, T. He, Y. Xie, F. Hu, J. Wu, Z. Luo, L. J. Fan, G. Shi, and Y. Zhu. Self-improving vision-language-action models with data generation via residual RL. *arXiv preprint arXiv:2511.00091*, 2025.

[25] M. Nakamoto, O. Mees, A. Kumar, and S. Levine. Steering your generalists: Improving robotic foundation models via value guidance. In *Conference on Robot Learning (CoRL)*, 2024.

[26] A. Wagenmaker, M. Nakamoto, Y. Zhang, S. Park, W. Yagoub, A. Nagabandi, A. Gupta, and S. Levine. Steering your diffusion policy with latent space reinforcement learning. *arXiv preprint arXiv:2506.15799*, 2025.

[27] H. Simsir and O. S. Oguz. Lagrangian perturbation diffusion steering: Latent reinforcement learning for generative policies. In *arXiv preprint arXiv:2606.01151*, 2026.

[28] A. Tang, W. Chen, A. Wagenmaker, C. Finn, and S. Levine. Improving robotic generalist policies via flow reversal steering. *arXiv preprint arXiv:2606.13675*, 2026.

[29] J. Luo, C. Xu, J. Wu, and S. Levine. Precise and dexterous robotic manipulation via human-in-the-loop reinforcement learning. *Science Robotics*, 10(105):eads5033, 2025.

[30] J. Lu, X. Qin, Y. Jiang, K. Wang, C. Zhang, B. Liang, J. Yang, M. Xu, and L. Zhao. UniSteer: Unified noise steering for efficient human-guided VLA adaptation. *arXiv preprint arXiv:2605.10821*, 2026.

[31] H. Deng, Y. Gao, Y. Lin, H. Liu, Z. Wu, and Z. Wang. Uniintervene: Agentic intervention for efficient real-world reinforcement learning. *arXiv preprint arXiv:2606.12372*, 2026.

[32] H. Chen, Z. Zou, C. Ma, Y. Pu, H. Zhang, Y. Chen, and Y. Yang. Accelerating robotic reinforcement learning with agent guidance. *arXiv preprint arXiv:2602.11978*, 2026.

[33] N. Carion, L. Gustafson, Y.-T. Hu, S. Debnath, R. Hu, D. Suris Coll-Vinent, C. Ryali, K. V. Alwala, H. Khedr, A. Huang, et al. Sam 3: Segment anything with concepts. In *International conference on learning representations*, volume 2026, pages 138846–138923, 2026.

[34] M. Sundermeyer, A. Mousavian, R. Triebel, and D. Fox. Contact-graspnet: Efficient 6-dof grasp generation in cluttered scenes. In *2021 IEEE international conference on robotics and automation (ICRA)*, pages 13438–13444. IEEE, 2021.

[35] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone. Libero: Benchmarking knowledge transfer for lifelong robot learning. *Advances in Neural Information Processing Systems*, 36:44776–44791, 2023.

[36] S. Ross, G. Gordon, and D. Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the fourteenth international conference on artificial intelligence and statistics*, pages 627–635. JMLR Workshop and Conference Proceedings, 2011.

[37] N. Rajaraman, Y. Han, L. Yang, J. Liu, J. Jiao, and K. Ramchandran. On the value of interaction and function approximation in imitation learning. *Advances in Neural Information Processing Systems*, 34:1325–1336, 2021.

[38] M. Oquab, T. Darcet, T. Moutakanni, H. Vo, M. Szafraniec, V. Khalidov, P. Fernandez, D. Haziza, F. Massa, A. El-Nouby, et al. Dinov2: Learning robust visual features without supervision. *arXiv preprint arXiv:2304.07193*, 2023.

[39] X. Chen, C. Wang, Z. Zhou, and K. Ross. Randomized ensembled double q-learning: Learning fast without a model. *arXiv preprint arXiv:2101.05982*, 2021.

A Implementation Details

A.1 Pose-to-Action-Chunk Conversion

At each state it chooses to correct, the diagnosis agent (Section 3.1) outputs a short-term target end-effector pose (p_g, R_g) and a gripper command $\gamma_g \in \{-1, +1\}$. The policy's action space is a 7-D OSC-delta command whose first six dimensions lie in $[-1, 1]$: at each control step, the LIBERO environment scales the action's translational and rotational blocks by per-axis limits L_{pos} and L_{rot} respectively, treats the scaled rotational block as an axis-angle delta, and applies both to the current end-effector pose to form a commanded target. We convert (p_g, R_g, γ_g) into a length-C chunk of such actions.

We empirically measure that, in the collision-free regime, an action a moves the end-effector by approximately $a \cdot S_{\text{pos}}$ meters and $a \cdot S_{\text{rot}}$ radians per step. Using this linear model as a forward predictor, we iteratively compute the remaining pose error in commanded-step units (dividing by L), clip the translational and rotational blocks independently to $[-1, 1]$, and advance an open-loop pose estimate by S; see Algorithm 2. The gripper command is concatenated onto each row to produce $\hat{a} \in \mathbb{R}^{C\times 7}$.

This open-loop heuristic does not model contact forces or long-horizon tracking error, so it is reliable only for short chunks and unobstructed paths; the agent's prompt makes both limitations explicit. The same forward model powers the `visualize_traj` tool that the agent uses to preview the end-effector's predicted path during reasoning, so the trajectory the agent visually verifies is the trajectory implied by the emitted chunk.

Algorithm 2 Pose-to-action-chunk conversion.

function POSETOCHUNK($p_g,\ R_g,\ p_0,\ R_0,\ C$)
  Input: target pose $(p_g,\ R_g)$; starting end-effector pose $(p_0,\ R_0)$; chunk length C
  Output: 6-D action delta sequence $\Delta \in \mathbb{R}^{C\times 6}$
  for $i = 0,\ 1,\ \ldots,\ C-1$ **do**
    $\delta_{\text{pos}} \leftarrow (p_g - p_i) \,/\, L_{\text{pos}}$ ▷ remaining error in units of one commanded step
    $\delta_{\text{rot}} \leftarrow \text{AxisAngle}(R_g\, R_i^{\top}) \,/\, L_{\text{rot}}$
    $a_{\text{pos}} \leftarrow \delta_{\text{pos}} \,/\, \max\big(1,\ \max_j |\delta_{\text{pos},j}|\big)$ ▷ independently clip each block to $[-1, 1]$
    $a_{\text{rot}} \leftarrow \delta_{\text{rot}} \,/\, \max\big(1,\ \max_j |\delta_{\text{rot},j}|\big)$
    $\Delta_i \leftarrow [\, a_{\text{pos}},\ a_{\text{rot}} \,]$
    $p_{i+1} \leftarrow p_i + a_{\text{pos}} \cdot S_{\text{pos}}$ ▷ advance open-loop pose estimate with empirical scales
    $R_{i+1} \leftarrow \text{AxisAngleToMat}(a_{\text{rot}} \cdot S_{\text{rot}}) \cdot R_i$
  end for
  return Δ
end function

A.2 RL Algorithm Details

State featurization and networks. Each camera image is encoded by a frozen DINOv2-small [38] into a 384-D embedding and concatenated with 8-D proprioception (end-effector position, axis-angle orientation, and gripper qpos) to form the RL state. The actor $\pi_\theta(\cdot \mid s, \tilde{a})$ is a two-layer MLP ($[512, 512]$, ReLU, $\tanh$ output) modeling a Gaussian with fixed $\sigma = 0.1$. The critic $Q_\psi(s, a)$ is a three-layer MLP ($[768, 512, 256]$, ReLU, LayerNorm) with a REDQ ensemble [39] of $N{=}10$ heads; the TD target takes the minimum over 2 randomly sampled members.

Reference-action dropout. Following RLT [7], we zero the reference chunk for 50% of training transitions so the actor sees $(s, \mathbf{0})$ instead of $(s, \tilde{a})$. We empirically found it helpful to exclude proposal rows from this mask.

Balanced sampling of proposal transitions. We sample each training batch with a fixed $50\%/50\%$ ratio of proposal to non-proposal transitions, to ensure that proposal states are sufficiently trained on.

Table 6: RL hyperparameters.

Description	**Value**
Chunk length C	10
Discount γ, per low-level step	0.999
Target-network Polyak coefficient τ	0.005
Actor Adam learning rate	10^{-4}
Critic Adam learning rate	3×10^{-4}
Actor Adam (β_1, β_2) / weight decay	$(0.9,\ 0.999)$ / 10^{-6}
Critic Adam (β_1, β_2) / weight decay	$(0.95,\ 0.999)$ / 10^{-6}
Batch size	128
Actor gradient updates per env step	1
Critic gradient updates per env step	10
Replay buffer capacity (chunk transitions)	10^5
Warmup episodes (execute $\tilde{a}$ verbatim)	20
Learning-starts threshold (transitions)	500

B Additional Experiment Results

Ablation on the number of proposals per diagnosis. Our default implementation places no explicit cap on the number of corrective proposals the diagnosis agent submits per failed episode, so the agent is free to submit a proposal at up to every chunk boundary in the trajectory. With chunk length $C{=}10$ and a 400-step episode horizon, this ceiling is $K{=}40$. We ablate this design by additionally training our method with caps $K \in \{1, 5, 10, 15, 20, 30\}$: the prompt instructs the agent that it may propose corrections at at most K of the candidate chunk boundaries. Table 7 summarizes the results.

Reducing K to 5 or 1 cuts output tokens from 36K to 22K and 14K, but throughput drops proportionally (0.88 and 0.70). Beyond $K{\approx}10$, throughput largely plateaus and even dips slightly at the uncapped default, while token cost continues to grow with K. We conjecture that proposals beyond the first $\sim$10 contribute little because the agent's additional corrections become less accurate, or because the remaining failure states are inherently difficult to correct. We default to no cap for simplicity of the algorithm.

Table 7: Effect of the per-diagnosis proposal cap K, averaged over 10 tasks at the end of the 400-episode budget. $K{=}40$ is the uncapped default used in the main results.

	$K{=}1$	$K{=}5$	$K{=}10$	$K{=}15$	$K{=}20$	$K{=}30$	$K{=}40$ (default)
Normalized Success Throughput	0.70 ± 0.02	0.88 ± 0.02	1.02 ± 0.02	0.99 ± 0.03	1.00 ± 0.02	1.01 ± 0.04	0.96 ± 0.05
Output Tokens / Diagnosis	14K	22K	28K	31K	33K	35K	36K
Proposals / Diagnosis	0.96	4.88	9.50	14.03	18.36	26.69	28.33

Ablation on the diagnoser model. We additionally evaluate our framework with three alternative diagnoser models: Claude Opus 4.8, GPT-6 Astra, and GPT-5.6 Sol, keeping all tools, prompts, and RL hyperparameters fixed. As shown in Table 8, Fable 5.1 achieves the highest throughput among the four models under the current framework, while the two GPT models are substantially more token-efficient, using roughly a third of the output tokens per diagnosis compared to Fable and Opus 4.8.

Table 8: Effect of the diagnoser model at the end of the 400-episode budget, averaged over 10 tasks and 6 seeds. Fable 5.1 is the default used in the main results.

	Fable 5.1 (default)	Opus 4.8	GPT-6 Astra	GPT-5.6 Sol
Normalized Success Throughput	0.96 ± 0.05	0.56 ± 0.02	0.59 ± 0.06	0.63 ± 0.05
Output Tokens / Diagnosis	36K	36K	13K	12K
Proposals / Diagnosis	28.33	4.33	26.51	4.96